

Session 7 — Smart Contract Programming II

Buổi 7 — Lập trình hợp đồng thông minh II

Token Standards / Chuẩn Token

Blockchain Technology and Cryptocurrency

TS. Nguyễn Trung Thành

ERC-20 & permit • ERC-721/1155 • on-chain metadata • upgradeability

Objectives / Mục tiêu

1. Read the **ERC-20** interface and explain the `approve` → `transferFrom` two-step and its risks
Đọc giao diện ERC-20 và giải thích hai bước approve/transferFrom cùng rủi ro
2. Build tokens on **OpenZeppelin v5** (extensions, AccessControl vs Ownable) and add **EIP-2612 permit**
Xây token trên OpenZeppelin v5 và thêm permit (EIP-2612)
3. Build an **ERC-721** NFT and serve **on-chain data: metadata** (no IPFS)
Xây NFT ERC-721 và phục vụ metadata on-chain (không IPFS)
4. Choose between **ERC-20 / 721 / 1155**, and reason about **upgradeability** (proxy, storage collision)
Chọn giữa 20/721/1155 và hiểu nâng cấp hợp đồng (proxy, storage collision)
5. Deploy both to **TrustKeys** and decode metadata off-chain (*triển khai & giải mã*)

Agenda — 3 hours / 3 giờ

Time	Content / Nội dung
0:00–0:55	Fungible tokens — ERC-20, SafeERC20, OpenZeppelin, permit
0:55–1:05	Break / <i>giải lao</i>
1:05–2:00	NFTs & upgradeability — ERC-721/1155, metadata, proxies
2:00–2:10	Break / <i>giải lao</i>
2:10–3:05	Lab <code>ClassToken</code> + <code>ClassBadge</code> on TrustKeys

Recap / Ôn lại (Session 6)

- A contract = **code** + **storage** at an address; `public` functions are its API
Hợp đồng = mã + storage tại một địa chỉ; hàm public là API
- **storage** is permanent & costly; **events** are cheap logs for the frontend
storage bền & đắt; event là log rẻ cho frontend
- We shipped `ClassRegistry` with the **Hardhat loop**: compile → test → deploy → read
Ta đã làm ClassRegistry với vòng lặp Hardhat

Today: reusable STANDARDS — money (ERC-20) and NFTs (ERC-721/1155).

Hôm nay: các CHUẨN tái dùng — tiền (ERC-20) và NFT (ERC-721/1155).

Hour 1 / Giờ 1

Fungible tokens / Token thay thế được

ERC-20, non-standard tokens, OpenZeppelin, permit

What is a token standard? / Chuẩn token là gì?

- A **shared interface** (function names + events) every wallet, DEX and explorer agrees on
Giao diện chung mọi ví, DEX, explorer đều hiểu
- The token is just a **contract with a balance ledger** — not a special coin type
Token chỉ là hợp đồng giữ sổ số dư — không phải loại coin đặc biệt
- **EIP** = the proposal; **ERC** = the resulting application standard (same number)
EIP là đề xuất; ERC là chuẩn ứng dụng (cùng số)
- Standardise the interface → **composability**: any app can plug in without custom code
Chuẩn hóa giao diện → khả năng ghép nối

Fungible = interchangeable units (1 CTK = 1 CTK). *Thay thế được: mọi đơn vị như nhau.*

ERC-20 (EIP-20) — the interface / Giao diện

```
function totalSupply() external view returns (uint256);
function balanceOf(address a) external view returns (uint256);
function transfer(address to, uint256 amt) external returns (bool);
function approve(address spender, uint256 amt) external returns (bool);
function allowance(address o, address s) external view returns (uint256);
function transferFrom(address from, address to, uint256 amt) external returns (bool);
event Transfer(address indexed from, address indexed to, uint256 value);
event Approval(address indexed owner, address indexed spender, uint256 value);
```

- Balances live in a `mapping(address => uint256)`; a transfer just **moves numbers**
Số dư trong mapping; chuyển tiền chỉ là đổi con số
- Standard since 2015 — the reason every wallet can show any token
Chuẩn từ 2015 — lý do mọi ví hiển thị được mọi token

`decimals` is display-only / chỉ để hiển thị

- On-chain, balances are **integers of base units** — there are no fractions in the EVM
On-chain, số dư là số nguyên đơn vị cơ sở — EVM không có phân số
- `decimals()` (usually **18**) tells the **UI** where to put the point
decimals (thường 18) bảo GIAO DIỆN đặt dấu phẩy ở đâu
- Balance `1_000000000000000000000000` with 18 decimals = **1.0 token**
Số dư đó với 18 decimals = 1,0 token
- ⚠ USDC/USDT use **6** decimals — assuming 18 everywhere is a classic bug
USDC/USDT dùng 6 decimals — mặc định 18 là lỗi kinh điển

`decimals` **changes NOTHING on-chain — it is metadata for humans.**

decimals KHÔNG đổi gì on-chain — chỉ là metadata cho người đọc.

Two ways to move tokens / Hai cách chuyển token

- `transfer(to, amt)` — *you* push your own tokens. Simple, direct.
Bạn tự đẩy token của mình. Đơn giản.
- `approve(spender, amt)` + `transferFrom(from, to, amt)` — you authorise a **spender contract** to *pull* up to an **allowance**
Bạn cấp quyền cho hợp đồng KÉO token tới mức allowance
- Why two steps? A plain `transfer` to a contract **runs no code there** — the contract can't react inside your transaction
Vì sao? transfer tới hợp đồng KHÔNG chạy code — không phản ứng được trong giao dịch của bạn

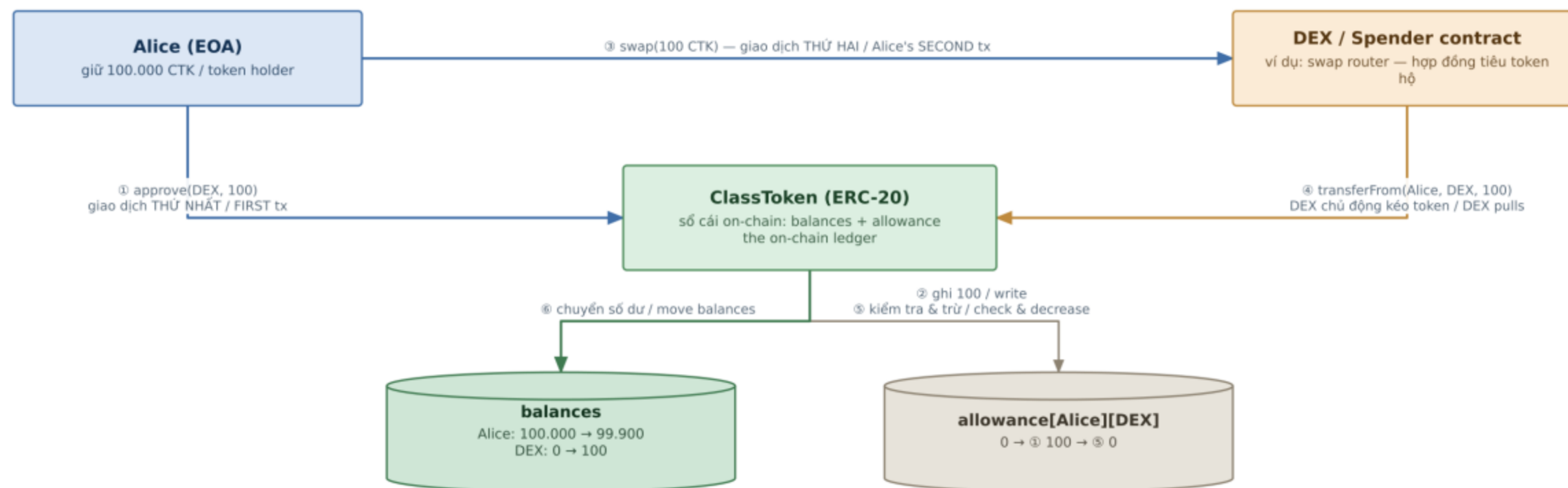
`transferFrom` **is how a DEX swaps your tokens inside one tx.**

transferFrom là cách DEX hoán đổi token ngay trong một giao dịch.

approve → transferFrom / Sơ đồ hai bước

ERC-20: approve → transferFrom — vì sao cần hai bước • why the two-step exists

Hợp đồng không thể 'kéo' token của bạn nếu chưa được cấp allowance • A contract can only pull your tokens up to the allowance you granted



Vì sao hai bước? transfer() tới hợp đồng chỉ đổi số dư — hợp đồng KHÔNG có code nào chạy, không biết ai gửi để đáp lại. transferFrom cho phép DEX kéo token NGAY TRONG giao dịch swap. Why 2 steps? A plain transfer() runs no code on the recipient; transferFrom lets the DEX pull tokens inside the swap tx itself.

⚠ INFINITE APPROVAL: approve(DEX, $2^{256}-1$) — tiện (khỏi approve lại) nhưng nếu DEX bị hack, kẻ tấn công rút SẠCH ví bạn mà không cần thêm chữ ký nào. Convenient, but a hacked spender drains your whole balance. Buổi 1: 'quản lý cấp quyền' • Buổi 8: các vụ hack approval thật.

The allowance / Hạn mức cấp quyền

- `allowance[owner][spender]` is a **second ledger**: how much a spender may pull
allowance là số thứ hai: spender được kéo bao nhiêu
- `transferFrom` **checks and decreases** the allowance, then moves balances
transferFrom kiểm tra & trừ allowance, rồi chuyển số dư
- Set it to what you need; the DEX pulls exactly that at swap time
Đặt đúng mức cần; DEX kéo đúng lượng đó khi swap

Mental model / Mô hình: approve = signing a *spending limit* for a contract.
approve = ký một hạn mức chi tiêu cho hợp đồng.

⚠ **Infinite approval risk / Rủi ro cấp quyền vô hạn**

- Many dApps ask for `approve(spender, 2256-1)` so you never re-approve
Nhiều dApp xin approve tối đa để khỏi approve lại
- Convenient — but if that spender is **hacked or malicious**, it can drain your **entire** balance with **no new signature**
Tiện — nhưng nếu spender bị hack, nó rút SẠCH ví, không cần chữ ký mới
- Real losses: countless "approval-drainer" phishing incidents
Thiệt hại thật: vô số vụ lừa "approval-drainer"
- Hooks back to **Session 1**: manage/revoke approvals; forward to **Session 8**: approval attacks
Nối Buổi 1 (quản lý cấp quyền) và Buổi 8 (tấn công approval)

Habit / Thói quen: approve the amount you need; periodically **revoke**.

Cấp đúng lượng cần; định kỳ thu hồi.

⚠ The approve race / Bẫy khi đổi allowance

- Changing a **non-zero** allowance to another **non-zero** value has a known race: a spender can front-run and spend the **old** allowance, then the **new** one
Đổi allowance khác 0 sang giá trị khác 0 có thể bị front-run: tiêu allowance CŨ rồi tiêu MỚI
- Safe habit: set to **0 first**, then to the new value — or use `permit`
An toàn: đặt về 0 trước, rồi mới đặt giá trị mới — hoặc dùng permit
- OpenZeppelin **v5 removed** `increaseAllowance` / `decreaseAllowance` to stop encouraging unsafe patterns
OZ v5 đã BỎ increase/decreaseAllowance để tránh khuyến khích cách làm không an toàn

Minting & burning / Phát hành & đốt

- `_mint(to, amt)` creates new supply; guard it with access control (`onlyOwner` / a role)
_mint tạo cung mới; canh bằng kiểm soát truy cập
- `_burn(from, amt)` destroys supply; `ERC20Burnable` exposes a public `burn()`
_burn hủy cung; ERC20Burnable mở hàm burn() công khai
- `totalSupply()` tracks mints – burns; `ERC20Capped` bounds it
totalSupply = mint – burn; ERC20Capped chặn trần

Our lab mints the full cap at deploy → a fixed supply, no later inflation.

Lab mint toàn bộ cap lúc deploy → cung cố định, không lạm phát về sau.

Non-standard tokens & SafeERC20 / Token lịch chuẩn

- ERC-20 says `transfer/approve` **return a bool** — but **USDT and others return nothing**

Chuẩn nói trả về bool — nhưng USDT không trả gì

- Code that does `require(token.transfer(...))` **reverts** on those tokens

Code kiểm tra giá trị trả về sẽ revert với các token đó

- OpenZeppelin **SafeERC20** wraps calls (`safeTransfer` , `safeTransferFrom` , `forceApprove`) and handles the missing/false return

SafeERC20 bọc lời gọi và xử lý trường hợp không trả/trả false

Rule / Nguyên tắc: when handling *arbitrary* tokens, always use **SafeERC20** .

Khi xử lý token bất kỳ, luôn dùng SafeERC20.

Don't reinvent it — OpenZeppelin v5 / Đừng viết lại

- **OpenZeppelin Contracts** = audited, community-standard implementations
OpenZeppelin = bản cài đặt đã kiểm toán, chuẩn cộng đồng
- You **inherit and compose** — less code, less attack surface, gas-reviewed
Bạn kế thừa & ghép — ít mã, ít lỗ hổng, đã tối ưu gas
- `wizard.openzeppelin.com` generates a starting contract by checkbox
Wizard sinh hợp đồng khởi đầu bằng tick chọn

```
import {ERC20} from "@openzeppelin/contracts/token/ERC20/ERC20.sol";
contract ClassToken is ERC20 {
    constructor() ERC20("ClassToken", "CTK") { _mint(msg.sender, 1_000_000e18); }
}
```

Useful ERC-20 extensions / Các extension hữu ích

Extension	What it adds / Thêm gì
ERC20Capped	hard max supply ceiling / trần tổng cung
ERC20Burnable	holders can burn their tokens / đốt token
ERC20Pausable	pause transfers in emergencies / tạm dừng chuyển
ERC20Permit	gasless approval by signature (EIP-2612) / cấp quyền bằng chữ ký

Compose only what you need — every extension is more code to trust.
Chỉ ghép cái cần — mỗi extension là thêm mã phải tin.

Who can mint? **AccessControl** vs **Ownable** / Ai được mint?

- **Ownable** — a single **owner** ; **onlyOwner** gates admin functions. Simple.
Ownable: một owner duy nhất; đơn giản
- **AccessControl** — named **roles** (**MINTER_ROLE** , **PAUSER_ROLE**), many holders, grant/revoke
AccessControl: nhiều vai trò, nhiều người giữ, cấp/thu hồi
- Use **Ownable** for a class demo; **AccessControl** when duties are split across a team/DAO
Ownable cho demo; AccessControl khi chia quyền cho nhóm/DAO

A single owner key is a single point of failure — mind who holds it.

Một khóa owner là điểm hỏng duy nhất — để ý ai giữ nó.

EIP-2612 `permit` — gasless approval / Cấp quyền không tốn gas

- Problem: `approve` is a **separate transaction** (extra gas + a second click)
Vấn đề: approve là giao dịch riêng (thêm gas + một cú click)
- `permit(owner, spender, value, deadline, v, r, s)` sets the allowance from an **off-chain signature**
permit đặt allowance từ chữ ký off-chain
- The **holder signs** (free); a **relayer submits** and pays the gas
Chủ token KÝ (miễn phí); relayer GỬI và trả gas
- Signature format = **EIP-712** typed data (hooks back to **Session 3**: `ecrecover`)
Chữ ký theo EIP-712 (nối Buổi 3: ecrecover)

Why permit is safe to submit / Vì sao permit an toàn

The signed message binds the approval to a specific context:

Thông điệp ký ràng buộc phê duyệt vào một ngữ cảnh cụ thể:

- **nonce** — a per-owner counter; each permit consumed **once** (no replay)
nonce: bộ đếm mỗi chủ; mỗi permit dùng MỘT lần (chống phát lại)
- **deadline** — the signature **expires** (*chữ ký hết hạn*)
- **chainId** + **contract address** (EIP-712 domain) — can't be replayed on another chain/contract
chainId + địa chỉ hợp đồng: không phát lại sang chain/hợp đồng khác

The contract recovers the signer on-chain and checks `signer == owner` .

Hợp đồng phục hồi người ký on-chain và kiểm tra `signer == owner`.

Break 10'

Next / *Tiếp theo*: NFTs (ERC-721/1155), on-chain metadata, and upgradeability.

NFT, metadata on-chain, và nâng cấp hợp đồng.

Hour 2 / Giờ 2

NFTs & upgradeability / NFT & nâng cấp

ERC-721, on-chain metadata, ERC-1155, proxies

ERC-721 (EIP-721) — non-fungible / Không thay thế được

- Each token has a **unique** `tokenId`; ownership is `ownerOf(tokenId)`
Mỗi token có tokenId riêng; sở hữu = ownerOf(tokenId)
- `balanceOf(owner)` = **count** of NFTs held (not an amount)
balanceOf = SỐ LƯỢNG NFT giữ (không phải số tiền)
- Transfers: `transferFrom` and `safeTransferFrom`; approvals per-token (`approve`) or for all (`setApprovalForAll`)
Chuyển: transferFrom / safeTransferFrom; cấp quyền từng token hoặc toàn bộ

Fungible ⇒ "how much"; **non-fungible** ⇒ "which one".

Thay thế được hỏi "bao nhiêu"; không thay thế được hỏi "cái nào".

What "safe" means / "safe" nghĩa là gì

- `_safeMint` / `safeTransferFrom` check: **if the recipient is a contract**, it must implement `onERC721Received`
Nếu người nhận là hợp đồng, nó phải cài onERC721Received
- If it doesn't, the transfer **reverts** — the NFT is not stranded in a contract that can't move it
Nếu không, revert — NFT không bị kẹt trong hợp đồng không thể chuyển nó
- This is a **handshake**, not a security guarantee about intent
Đây là cái bắt tay, không bảo đảm ý định

Discovery: `ERC-165 supportsInterface(id)` lets callers ask "do you implement ERC-721?"

ERC-165 cho phép hỏi "bạn có cài ERC-721 không?"

tokenURI & the metadata JSON / Metadata JSON

`tokenURI(tokenId)` returns a **URI** pointing to a JSON document:

tokenURI trả về URI trỏ tới tài liệu JSON:

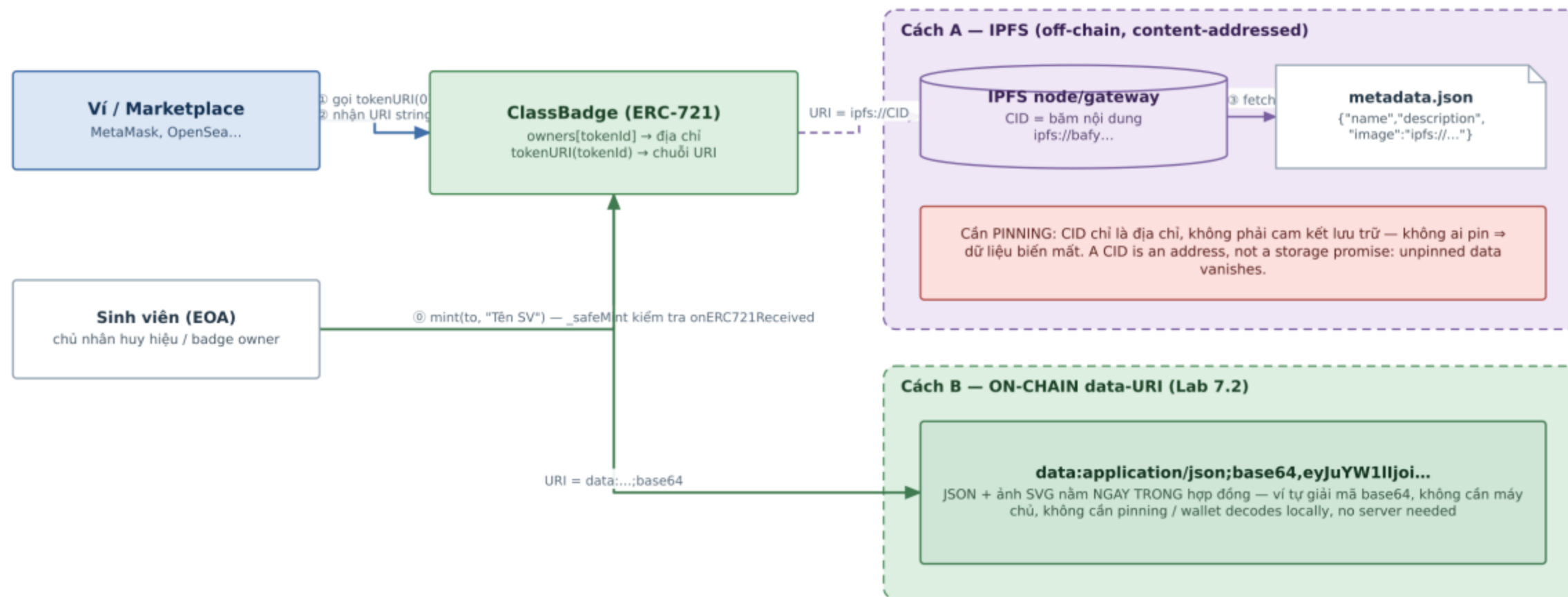
```
{
  "name": "Blockchain Class Badge #0",
  "description": "On-chain proof of attendance ...",
  "image": "data:image/svg+xml;base64,PHN2Zy...",
  "attributes": [ { "trait_type": "Student", "value": "Trần Thị B" } ]
}
```

- The chain stores **a pointer** — where the JSON + image *live* is a design choice
Chuỗi lưu con trỏ — JSON & ảnh sống ở đâu là lựa chọn thiết kế

Where does metadata live? / Metadata sống ở đâu?

ERC-721: mint & giải nghĩa tokenURI — IPFS vs metadata on-chain (data-URI)

Token on-chain chỉ lưu con trỏ URI — ảnh & JSON sống ở đâu là lựa chọn thiết kế • The chain stores a URI pointer; where JSON+image live is a design choice



TrustKeys testnet không có gateway/pinning IPFS ⇒ Lab dùng Cách B: metadata sống mãi cùng hợp đồng. Đánh đổi: on-chain = bất biến + tốn gas theo kích thước; IPFS = rẻ + phải duy trì pinning.
Trade-off: on-chain = immutable but gas-priced per byte; IPFS = cheap but needs pinning discipline.

IPFS vs on-chain / IPFS so với on-chain

	IPFS (off-chain)	On-chain <code>data:</code> URI
Address	<code>ipfs://CID</code> (hash of content)	<code>data:application/json;base64,...</code>
Storage	a node must pin it	in the contract forever
Cost	cheap bytes	gas per byte
Risk	unpinned ⇒ data vanishes	large ⇒ expensive to deploy
Needs a server?	yes (gateway/pinning)	no

TrustKeys has no IPFS gateway ⇒ our lab uses the on-chain `data:` URI.

TrustKeys không có IPFS ⇒ lab dùng data-URI on-chain.

On-chain metadata in Solidity / Metadata on-chain

```
bytes memory json = abi.encodePacked(
    '{"name":"Badge #', tokenId.toString(),
    '", "image":"data:image/svg+xml;base64,', Base64.encode(svgBytes),
    '", "attributes":[{"trait_type":"Student", "value":"', name, '"}]}'
);
return string(abi.encodePacked(
    "data:application/json;base64,", Base64.encode(json)));
```

- OZ **Base64** + **Strings** build it; a wallet **decodes locally** — no server
Base64 + Strings dựng; ví tự giải mã — không cần máy chủ
- Immutable and self-contained — perfect for a permanent class badge
Bất biến & tự chứa — hợp cho huy hiệu vĩnh viễn

Advisory extras / Phần mở rộng bổ trợ

- **EIP-2981 royalties** — `royaltyInfo(tokenId, salePrice)` returns *(receiver, amount)*.

It is **advisory** — marketplaces choose to honour it

EIP-2981: trả về người nhận + mức phí; chỉ khuyến nghị, chợ tự quyết tôn trọng

- **EIP-5192 soulbound** — a minimal "locked" (non-transferable) NFT for credentials/badges

EIP-5192: NFT "khóa", không chuyển nhượng — hợp cho chứng chỉ/huy hiệu

On-chain code can't force off-chain marketplaces to pay royalties.

Mã on-chain không ép được chợ off-chain trả royalty.

ERC-1155 — multi-token / Đa token

- **One contract, many token ids** — each id can be fungible **or** non-fungible
Một hợp đồng, nhiều id — mỗi id có thể thay thế được hoặc không
- **Batch** ops: `balanceOfBatch`, `safeBatchTransferFrom` — mint/move many ids in **one tx** (cheaper)
Thao tác theo lô: chuyển nhiều id trong một giao dịch (rẻ hơn)
- Great for **games / editions**: gold (fungible) + swords (unique) in the same contract
Hợp cho game/phiên bản: vàng (đồng nhất) + kiếm (độc nhất) chung một hợp đồng

Metadata via a URI template with `{id}` — one pattern for all tokens.

Metadata theo mẫu URI có `{id}` — một khuôn cho mọi token.

Which standard? / Chọn chuẩn nào?

Need / Nhu cầu	Use / Dùng
Currency, points, shares / <i>tiền, điểm, cổ phần</i>	ERC-20
Unique 1-of-a-kind items / <i>vật phẩm độc nhất</i>	ERC-721
Many item types, batch, games / <i>nhiều loại, theo lô, game</i>	ERC-1155
Non-transferable credential / <i>chứng chỉ không chuyển</i>	721 + EIP-5192

Rule of thumb / Kinh nghiệm: "how much" → 20; "which one" → 721; "a catalogue" → 1155.

"bao nhiêu" → 20; "cái nào" → 721; "một danh mục" → 1155.

Upgradeability — why & why-not / Nâng cấp — nên & không

- Contracts are **immutable**: a bug is permanent, so teams want an **upgrade path**
Hợp đồng bất biến: lỗi tồn tại vĩnh viễn, nên muốn đường nâng cấp
- But upgradeability = **admin power to change the code users trust** — a centralization & attack risk
Nhưng nâng cấp = quyền đổi mã người dùng tin — rủi ro tập trung & tấn công
- Alternatives: keep it **immutable + migrate** to a new contract; or govern upgrades via a **timelock/DAO**
Cách khác: bất biến + di trú; hoặc quản trị nâng cấp bằng timelock/DAO

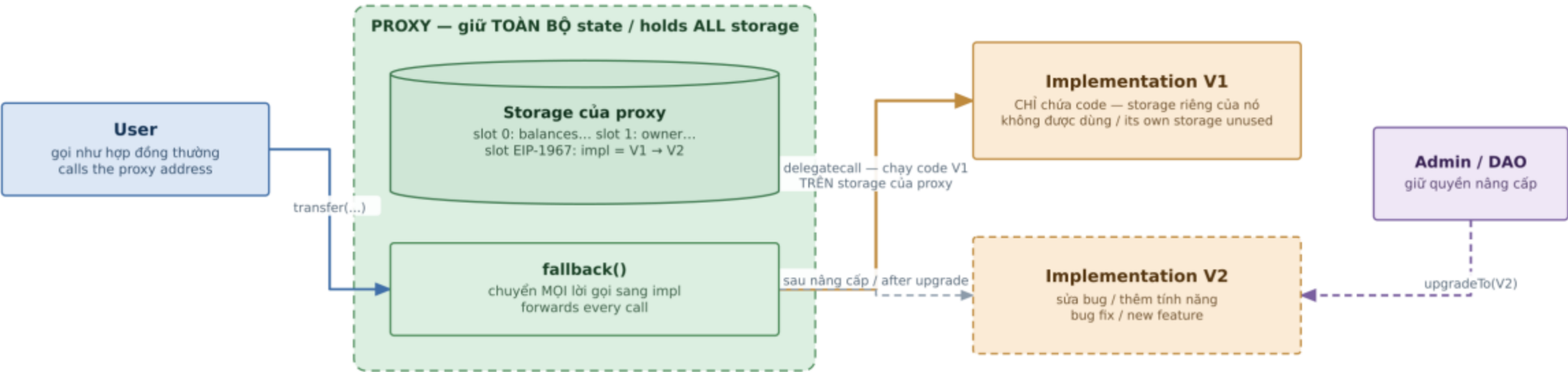
"Upgradeable" means someone can rewrite the rules — know who.

"Nâng cấp được" nghĩa là ai đó viết lại luật — phải biết là ai.

How a proxy works / Proxy hoạt động ra sao

Proxy + delegatecall: nâng cấp hợp đồng — và cái bẫy storage collision

Code nằm ở implementation, STATE nằm ở proxy — đổi con trỏ code là 'nâng cấp' • Code lives in the implementation, STATE lives in the proxy; upgrading = repointing



Transparent proxy: hàm upgrade nằm Ở PROXY (admin bị tách khỏi lời gọi thường). UUPS: hàm upgrade nằm Ở IMPLEMENTATION + `_authorizeUpgrade` (rẻ gas hơn — quyền override = khóa vĩnh viễn). Transparent: upgrade logic in proxy • UUPS: in the implementation itself.

Constructor KHÔNG chạy trên storage của proxy ⇒ dùng `initialize()` + modifier initializer (gọi đúng MỘT lần). Constructors never touch proxy storage — use an initializer, and protect it so it runs exactly once.

⚠ STORAGE COLLISION: V2 chen biến mới lên đầu / đổi thứ tự khai báo ⇒ slot lệch ⇒ V2 đọc 'owner' ra giá trị 'balances' — state hỏng IM LẶNG, không revert. Quy tắc: chỉ THÊM biến vào CUỐI, dành sẵn storage gaps, và để OpenZeppelin Upgrades plugin kiểm tra layout tự động. Reordered slots corrupt state silently — append-only layout + storage gaps + OZ Upgrades checks.

delegatecall & the proxy split / Tách proxy

- `delegatecall` runs another contract's **code** on the **caller's storage**
delegatecall chạy CODE hợp đồng khác TRÊN storage của bên gọi
- **Proxy** holds **all state** + an `impl` pointer; **implementation** holds only **code**
Proxy giữ toàn bộ state + con trỏ impl; implementation chỉ giữ code
- **Upgrade = repoint** `impl` from V1 to V2 — same address, same state, new logic
Nâng cấp = trỏ lại impl từ V1 sang V2 — cùng địa chỉ, cùng state, logic mới

Transparent proxy: upgrade logic in the **proxy**. **UUPS**: in the **implementation**

(`_authorizeUpgrade`), cheaper — *forget the override* $\Rightarrow$ *locked forever*.

Transparent: logic nâng cấp ở proxy. UUPS: ở implementation, rẻ hơn — quên override = khóa vĩnh viễn.

The two proxy traps / Hai cái bẫy proxy

- **No constructor.** Constructors run on the *implementation's* storage, not the proxy → use an `initialize()` + `initializer` modifier (runs **exactly once**)
Không dùng constructor. Dùng initialize() + modifier initializer (chạy đúng một lần)
- **Storage collision.** V2 must keep the **same slot layout**: only **append** new variables, reserve **storage gaps** — reordering corrupts state **silently** (no revert)
Storage collision: giữ nguyên bố cục slot; chỉ THÊM biến vào cuối, chừa gap — đổi thứ tự làm hỏng state IM LẶNG

Let the OZ Upgrades plugin check the layout for you.

Để plugin OZ Upgrades kiểm tra bố cục giúp bạn.

Preview: ERC-4626 vault / Xem trước: kho ERC-4626

- A **standard tokenized vault**: deposit an ERC-20, receive **share tokens**; shares track your slice of the pool

Kho token hóa chuẩn: gửi ERC-20, nhận token phần; phần theo dõi tỉ lệ của bạn trong hồ

- Standardises `deposit / mint / withdraw / redeem` + share↔asset conversions

Chuẩn hóa deposit/mint/withdraw/redeem + quy đổi phần↔tài sản

- The backbone of many **DeFi** yield products — full treatment in **Session 10**

Xương sống của nhiều sản phẩm lợi suất DeFi — chi tiết ở Buổi 10



Break 10' → Lab

Have ready / *chuẩn bị*: MetaMask on TrustKeys, `npm i @openzeppelin/contracts@5.0.2`,
funded test address.

MetaMask trên TrustKeys, đã cài OZ 5.0.2, địa chỉ thử đã có tiền.

Lab 7 — ClassToken + ClassBadge (50') / Thực hành

Build, test, deploy two real token contracts to TrustKeys.

Xây, kiểm thử, triển khai hai hợp đồng token thật lên TrustKeys.

'	Step / Bước	Deliverable / Sản phẩm
5	setup + <code>npm i @openzeppelin/contracts@5.0.2</code>	project ready / <i>dự án sẵn sàng</i>
15	<code>ClassToken</code> (ERC-20 capped) + Chai tests	tests green / <i>test xanh</i>
15	<code>ClassBadge</code> (ERC-721, on-chain JSON) + tests	tokenURI decodes / <i>giải mã được</i>
10	deploy both; import CTK, send 100; decode badge	on-chain proof / <i>bằng chứng</i>
5	Bonus permit (EIP-712 off-chain sign)	permit test / <i>test permit</i>

Files / *tệp*: `lab/lab07_worksheet.md` , `lab/solutions/` , Remix single files.

Lab focus points / Điểm nhấn của lab

- **Pin** `@openzeppelin/contracts@5.0.2` + EVM `paris` — newer OZ needs Cancun `mcopay`, which TrustKeys' Geth won't run
Ghim OZ 5.0.2 + EVM paris — OZ mới cần opcode Cancun, TrustKeys không chạy
- The `_update` override is **required** (ERC20 + ERC20Capped both define it)
Bắt buộc override _update (ERC20 + ERC20Capped cùng định nghĩa)
- Test decodes the badge base64 in JS: `Buffer.from(b64, "base64")` → `JSON.parse`
Test giải mã base64 huy hiệu trong JS rồi JSON.parse
- Read state via **ethers** (no explorer): `balanceOf`, `tokenURI` view calls
Đọc trạng thái bằng ethers (không explorer)

🎓 Final Project — registration due this week / Đăng ký đồ án hạn tuần này

Opened in Session 6 — teams of 2-3, topic + team due end of week.

(Mở từ Buổi 6 — nhóm 2-3, đề tài + nhóm hạn cuối tuần.)

- If you haven't registered, submit the form **before Session 8**

Chưa đăng ký thì nộp biểu mẫu trước Buổi 8

- **Today's session directly feeds** token/NFT projects — Topics 1, 2, 6, 7 use exactly ERC-20/721/1155

Buổi hôm nay phục vụ trực tiếp các đề tài token/NFT — Đề tài 1, 2, 6, 7

- Next gate: **2-page design due Session 10** *(mốc sau: thiết kế 2 trang, Buổi 10)*

Link in the class channel — don't miss the deadline. *(Link trong kênh lớp — đừng lỡ hạn.)*

Homework / Bài về nhà (trước Buổi 8)

1. Finish `ClassToken` + `ClassBadge` : **all tests green**, deploy to TrustKeys, paste both addresses + a tx hash
Hoàn thiện: test xanh, deploy, dán 2 địa chỉ + 1 tx hash
2. Import CTK in MetaMask, send 100 CTK to a classmate; decode your badge `tokenURI`
(nhập CTK, gửi 100, giải mã badge)
3. Add the `permit` bonus test (off-chain sign, relayer submit) *(thêm test permit)*
4. Read / *Đọc*: eips.ethereum.org **EIP-20, EIP-721, EIP-1155, EIP-2612** •
docs.openzeppelin.com/contracts

Next / Buổi sau

Session 8 — Smart Contract Security

Re-entrancy • access control • approval attacks

You built tokens & NFTs; next you learn **how they get exploited — and hardened.**

Bạn vừa xây token & NFT; tiếp theo học cách chúng bị khai thác — và phòng thủ.

References / Tài liệu tham khảo

- **EIP-20** (ERC-20), **EIP-721** (NFT), **EIP-1155** (multi-token), **EIP-2612** (permit), **EIP-165** (interface detection), **EIP-2981** (royalties), **EIP-5192** (soulbound), **EIP-1967** (proxy slots), **EIP-4626** (vault) — eips.ethereum.org
- **OpenZeppelin Contracts** — docs.openzeppelin.com/contracts (ERC-20/721/1155, extensions, SafeERC20, Base64, AccessControl)
- **OpenZeppelin Upgrades** — docs.openzeppelin.com/upgrades-plugins (proxies, storage layout)
- **Contracts Wizard** — wizard.openzeppelin.com
- **Solidity docs** — docs.soliditylang.org • **Ethers v6** — docs.ethers.org
- **Ethereum dev portal** — ethereum.org/developers (tokens, ERC-721)
- **Remix (TrustKeys)** — remix.trustkeys.com • **TrustKeys L1**: RPC
`l1testnet.trustkeys.network`, chainId 11968